The following code uses the Integer.is_even/1 macro, so the require Integer must be added to the module:

```
defmodule ExampleModule do
  require Integer

  def print_when_even(number) do
    if Integer.is_even(number) do
      IO.puts "It's even!"
    end
  end
end
```

If you try to use Integer.is_even/1 without requiring it, you'll get an error.

```
** (CompileError) you must require Integer before invoking
    the macro Integer.is_even/1
```

Use

The use keyword serves as a hook point during the compilation of a module. The module that's "used" will be able to add code to the module with the use statement. There is no like-for-like comparison with Ruby because Elixir's use is executed at compile time, but it's most similar to how ActiveSupport::Concern works.

use is often used by some libraries to create a seamless developer experience. The next example shows what an Ecto Schema looks like. This example shows several functions that make up a schema-building DSL:

```
defmodule SmsMessage do
  use Ecto.Schema

  schema "sms_messages" do
    field :body, :string
    field :from, :string
    field :to, :string
  end
end
```

The schema/2 and field/3 functions are all defined in the Ecto.Schema module. The line use Ecto.Schema makes these functions available inside of the module. A lot of complex setup is involved in pulling this off, so import won't work here.

To truly understand use, you have to understand that Elixir is a compiled language. Let's dive into what this means.